

SC2001 Project 1: Hybrid Sort

PRESENTED BY TEAM 3

Lee Zi An

Mohamed Fazli Bin Mohd Yazid

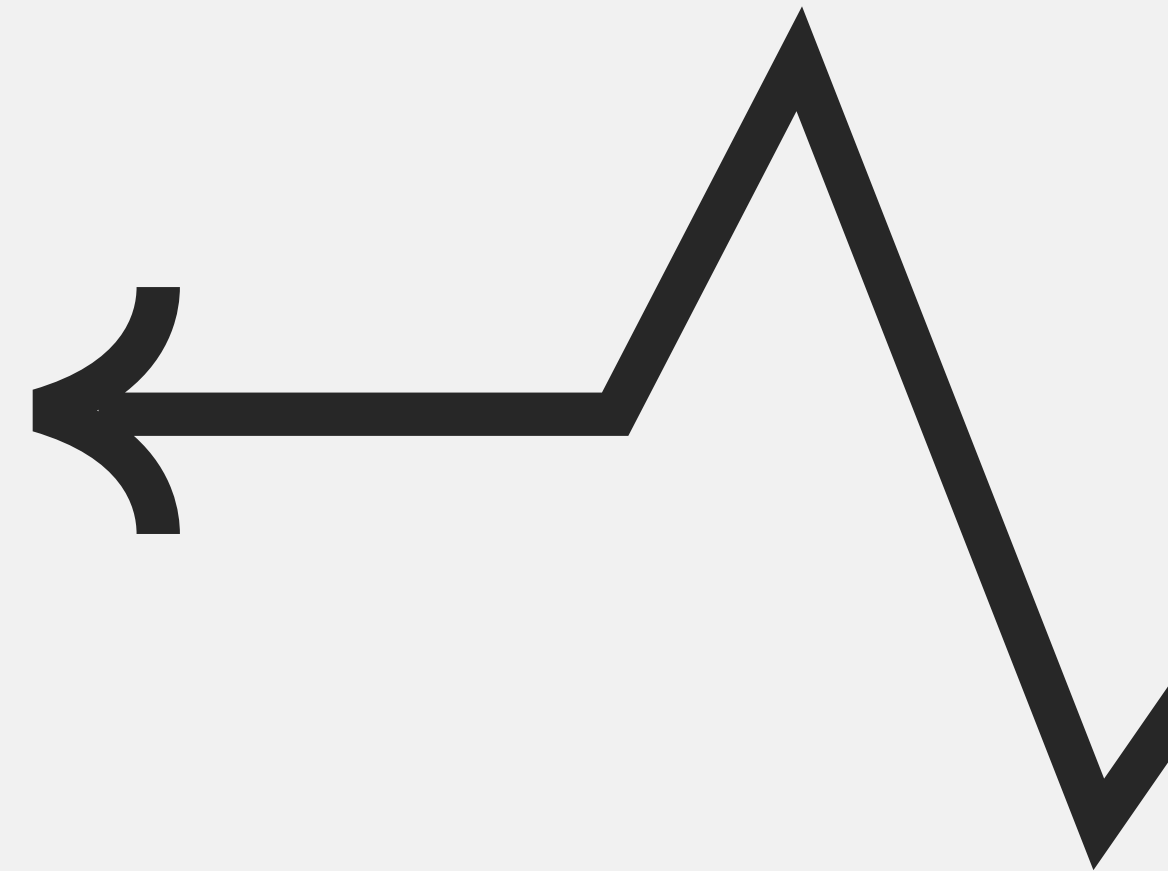
Muhammad Alfiq Bin Abdul Sabnie



Introduction



- Algorithm Implementation
- Generate input data
- Analyze time complexity
- Compare with original MergeSort



(a) Algorithm Implementation

insertion sort

hybrid sort

```
191
192 void hybrid_sort_recursion(int arr[], int L, int r, int S){
193     if (L < r){
194         if ((r-L) <= S){
195             insertion_sort(arr, L, r);
196         }
197         else{
198             int m = L + (r - L) / 2;
199             hybrid_sort_recursion(arr, L, m, S);
200             hybrid_sort_recursion(arr, m+1, r, S);
201             merge(arr, L, m, r);
202         }
203     }
204 }
205
```

```
25 int insertion_sort(int arr[], int n){
26     for (int i = 1; i < n; i++){
27         for (int j = i; j > 0; j--){
28             if (arr[j] < arr[j-1]){
29                 swap(arr, j, j-1);
30             }
31         }
32     }
33 }
```

mergesort

```
211 void mergesort_recursion(int arr[], int L, int r){
212     if (L < r){
213         int m = L + (r - L) / 2;
214         mergesort_recursion(arr, L, m);
215         mergesort_recursion(arr, m+1, r);
216         merge(arr, L, m, r);
217     }
218 }
```

(b) Generate input data

Generate random array of length n and max value x

```
287 int* generateArr(int n, int x){
288     /* generate a random arr of size n and max value x */
289     // https://stackoverflow.com/questions/43015080/creating-large-arrays-in-c
290     int* arr = (int *)malloc(n * sizeof(int));
291     for (int i = 0; i < n; i++){
292         int rand_number = (rand() % x) + 1;
293         arr[i] = rand_number;
294     }
295     return arr;
296 }
```

Generate an array of linearly spaced values

```
298 int* linspace(int start, int end, int num){
299     /* generate an array with linearly spaced values */
300     int* arr = (int *) malloc(num * sizeof(int));
301     int step = (end - start) / (num - 1);
302     for (int i = 0; i < num; i++){
303         arr[i] = start + (i * step);
304
305         // assign last value of arr to value end
306         if (i == (num - 1)){
307             arr[i] = end;
308         }
309     }
310     return arr;
311 }
```

Fixed S, Changing n

n values: use linear space to produce 100
n values from 1000-100000000

S values: for loop from 5-50 with intervals
of 5

for every **S** value, generate array with the
100 **n** values, and apply hybrid sort

write data into csv file

```
37 x = 10000000;  
38 start = 1000;  
39 stop = 10000000;  
40 num = 100;  
41 arr_n = linspace(start, stop, num);
```

```
23 int main(){  
55 printf("fixed: S, varying: n\n");  
56 for (int i = 25; i <= 50; i+=5){  
57     S = i;  
58     printf("S: %d\n", S);  
59  
60     // write to csv in c  
61     // https://dev.to/arepp23/how-to-write-to-a-csv-file-in-c-1l5b  
62     snprintf(filename, sizeof(filename), "data_c_i_new/hybridsort_n_1000000_S_%d.csv", S);  
63     printf("%s\n", filename);  
64     fpt = fopen(filename, "w+");  
65     fprintf(fpt, "n,x,S,runtime,comparison\n");  
66  
67     printf("-----\n");  
68     printf("%-8c | runtime (sec) | # of comparison \n", 'n');  
69     printf("-----\n");  
70  
71     for (int i = 0; i < num; i++){  
72         n = arr_n[i];  
73         arr = generateArr(n, x);  
74         comparison = 0;  
75  
76         begin = clock();  
77         hybrid_sort(arr, n, S);  
78         end = clock();  
79         runtime = (double) (end - begin) / CLOCKS_PER_SEC;  
80  
81         printf("%-8d | %-13f | %-15ld\n", n, runtime, comparison);  
82         fprintf(fpt, "%d,%d,%d,%f,%ld\n", n, x, S, runtime, comparison);  
83     }  
84     putchar('\n');  
85     fclose(fpt);  
86 }
```

```
S: 30  
data_c_i_new/hybridsort_n_1000000_S_30.csv  
-----  
n      | runtime (sec) | # of comparison  
-----  
1000   | 0.000186      | 18251  
102000 | 0.028503      | 2435241  
203000 | 0.047745      | 5037282  
304000 | 0.072248      | 6894736  
405000 | 0.111668      | 10443054  
506000 | 0.141529      | 14613565  
607000 | 0.160340      | 14364636  
708000 | 0.186584      | 17855033  
809000 | 0.219619      | 21657419
```

```
1 n,x,S,runtime,comparison  
2 1000,10000000,30,0.000186,18251  
3 102000,10000000,30,0.028503,2435241  
4 203000,10000000,30,0.047745,5037282  
5 304000,10000000,30,0.072248,6894736  
6 405000,10000000,30,0.111668,10443054  
7 506000,10000000,30,0.141529,14613565  
8 607000,10000000,30,0.160340,14364636  
9 708000,10000000,30,0.186584,17855033  
10 809000,10000000,30,0.219619,21657419
```

Fixed n, Changing S

```
97 printf("fixed: n, varying: S\n");
98 int arr_n_temp[] = {1000, 5000, 10000, 50000, 100000, 500000, 1000000, 2500000, 5000000, 7500000, 10000000};
99 for (int i = 0; i < sizeof(arr_n_temp)/sizeof(int); i++){
100     n = arr_n_temp[i];
101     arr = generateArr(n, x);
102     printf("n: %d\n", n);
103
104     snprintf(filename, sizeof(filename), "data_c_ii_new/hybridsort_n_%d.csv", n);
105     printf("%s\n", filename);
106     fpt = fopen(filename, "w+");
107     fprintf(fpt, "n,x,S,runtime,comparison\n");
108
109     printf("-----\n");
110     printf("%-8c | runtime (sec) | # of comparison\n", 'S');
111     printf("-----\n");
112
113     for (int i = 1; i <= 100; i++){
114         comparison = 0;
115         S = i;
116
117         begin = clock();
118         hybrid_sort(arr, n, S);
119         end = clock();
120         runtime = (double) (end - begin) / CLOCKS_PER_SEC;
121
122         printf("%-8d | %-13f | %-15ld\n", S, runtime, comparison);
123         fprintf(fpt, "%d,%d,%d,%f,%ld\n", n, x, S, runtime, comparison);
124     }
125     putchar('\n');
126     fclose(fpt);
127 }
```

S values: use a for loop from 1-100

n values: predefined values between 1,000-10,000,000

for every **n** value, apply hybrid sort with the different **S** value

write data to csv

```
> ./hybrid_merge_sort
fixed: n, varying: S
n: 1000
data_c_ii_new/hybridsort_n_1000.csv
-----
S      | runtime (sec) | # of comparison
-----
1      | 0.000086      | 8701
2      | 0.000051      | 5044
3      | 0.000047      | 5508
4      | 0.000072      | 5508
5      | 0.000055      | 5508
6      | 0.000050      | 5700
7      | 0.000042      | 6948
8      | 0.000039      | 6948
9      | 0.000038      | 6948
10     | 0.000031      | 6948
```

```
1  n,x,S,runtime,comparison
2  1000,10000000,1,0.000086,8701
3  1000,10000000,2,0.000051,5044
4  1000,10000000,3,0.000047,5508
5  1000,10000000,4,0.000072,5508
6  1000,10000000,5,0.000055,5508
7  1000,10000000,6,0.000050,5700
8  1000,10000000,7,0.000042,6948
9  1000,10000000,8,0.000039,6948
10 1000,10000000,9,0.000038,6948
```

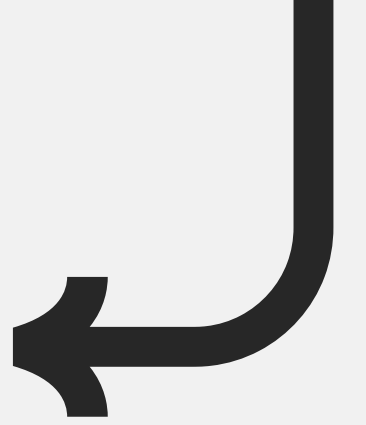


Analyzing time
complexity



Analyzing time complexity

Theoretical analysis of Insertion Sort Algorithm



Time Complexity

Insertion Sort (array size s)

Hybrid Sort

Best Case

$$O(s)$$

$$(n/s) * s = O(s)$$

Worst Case

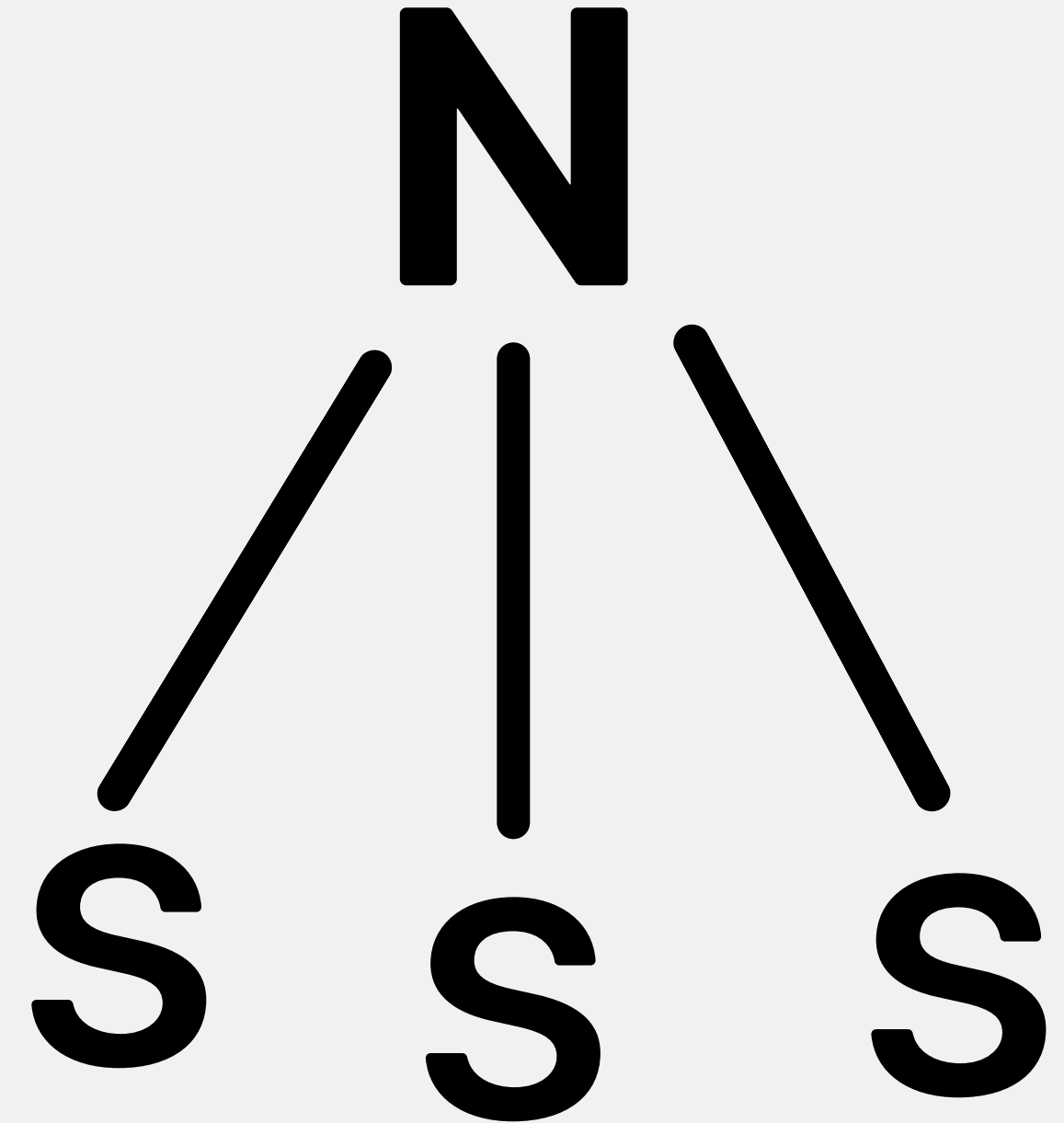
$$O(s^2)$$

$$(n/s) * s^2 = O(ns)$$

Part (c)(i): Analysis of time complexity

Understanding the formula

- Total cost of insertion sort for a single block of s element :
Best case : $O(s)$
Worst case : $O(s^2)$
- Total cost of applying insertion sort :
Best case : $(n/s)*s = O(n)$
Worst case : $(n/s)*s^2 = O(ns)$



Analysis of time complexity (Merging)

Understanding the formula

$$(2^i) * s = n$$

$$(2^i) = n/s$$

$$i * \log(2) = \log(n/s)$$

$$i = \log(n/s)$$

Taking log on both sides

No. of iteration

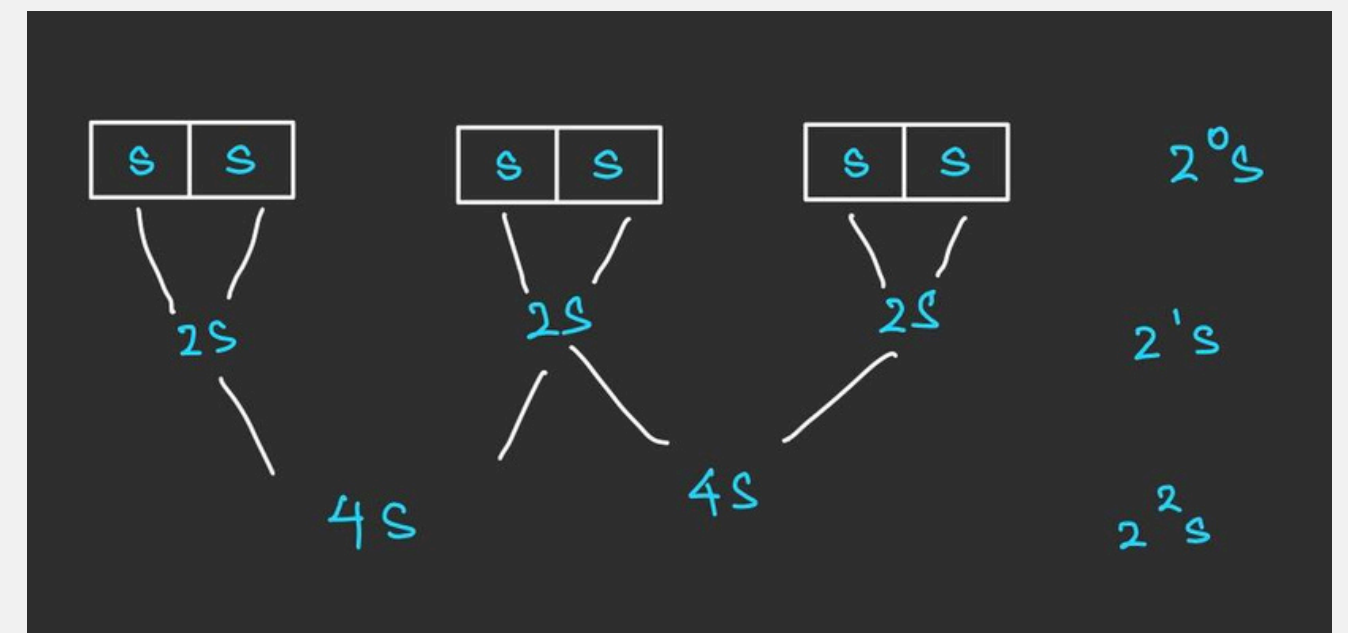
$$\text{Cost of merging} = O(n)$$

$$\text{Total cost of merging} = \text{No. of iteration} * \text{Cost of iteration}$$

$$= \log(n/s) * n$$

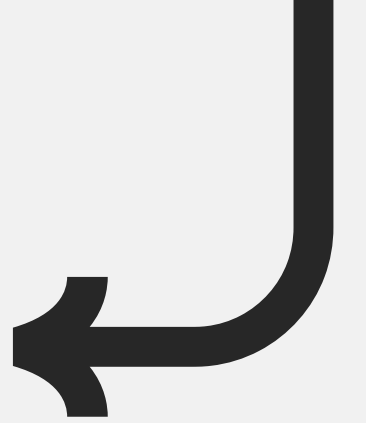
$$= n * \log(n/s)$$

$$= O(n * \log(n/s))$$



Analyzing time complexity

Theoretical analysis of Merge Sort Algorithm



Time Complexity

Merge Sort (array size s)

Hybrid Sort

Best Case

$O(n \cdot \log n)$

$O(n \cdot \log(n/s))$

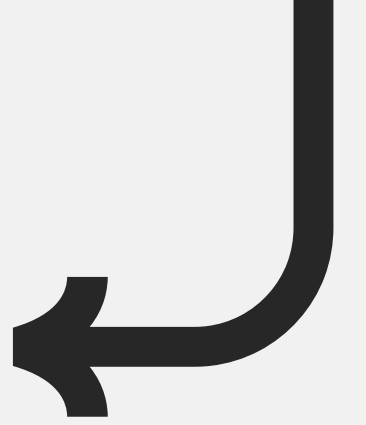
Worst Case

$O(n \cdot \log n)$

$O(n \cdot \log(n/s))$

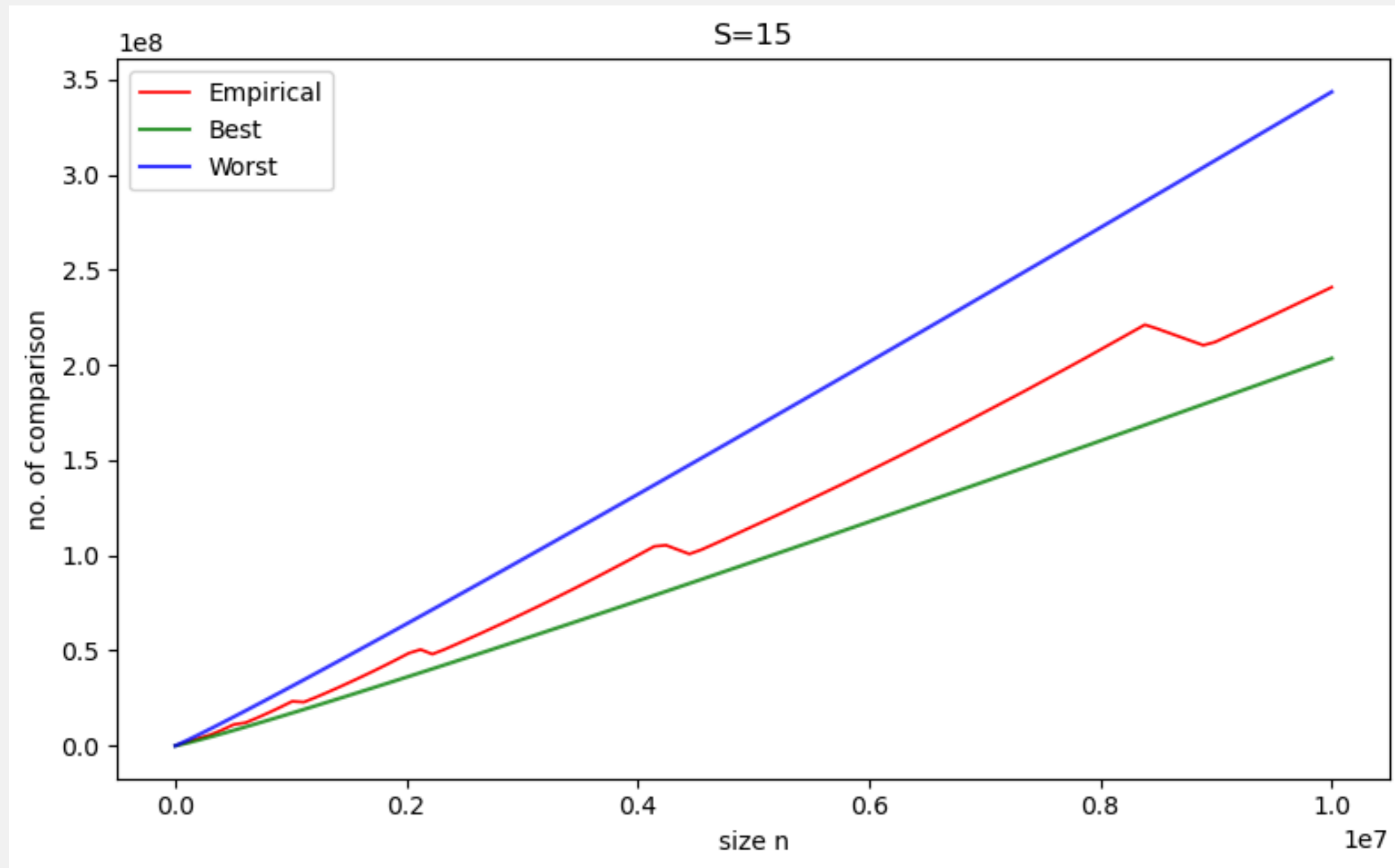
Analyzing time complexity

Theoretical analysis of Hybrid Sort Algorithm



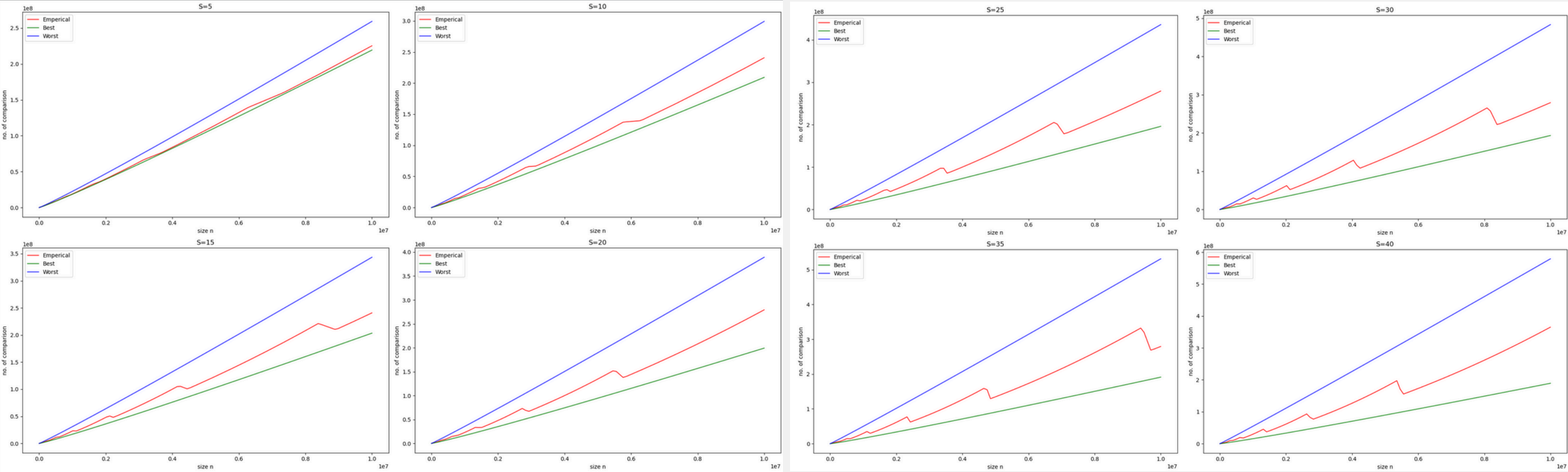
Time Complexity	Hybrid Sort (Insertion + Merge)	Hybrid Sort (Fixed s)
Best Case	$O(n + n \log(n/s))$	$O(n \log(n))$
Worst Case	$O(ns + n \log(n/s))$	$O(n \log(n))$

(ci) compare empirical vs theoretical

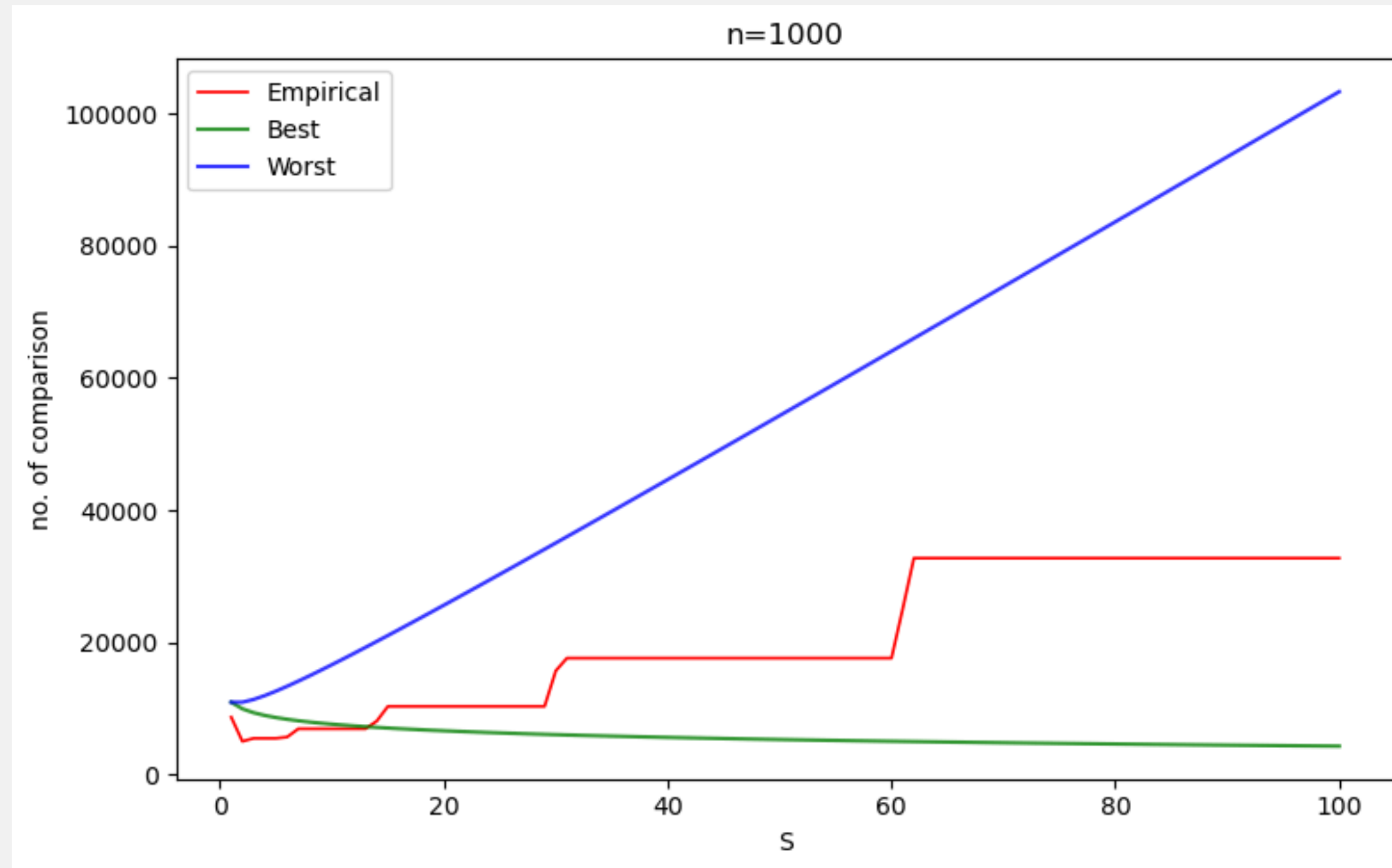


empirical data - average case
between best and worst case

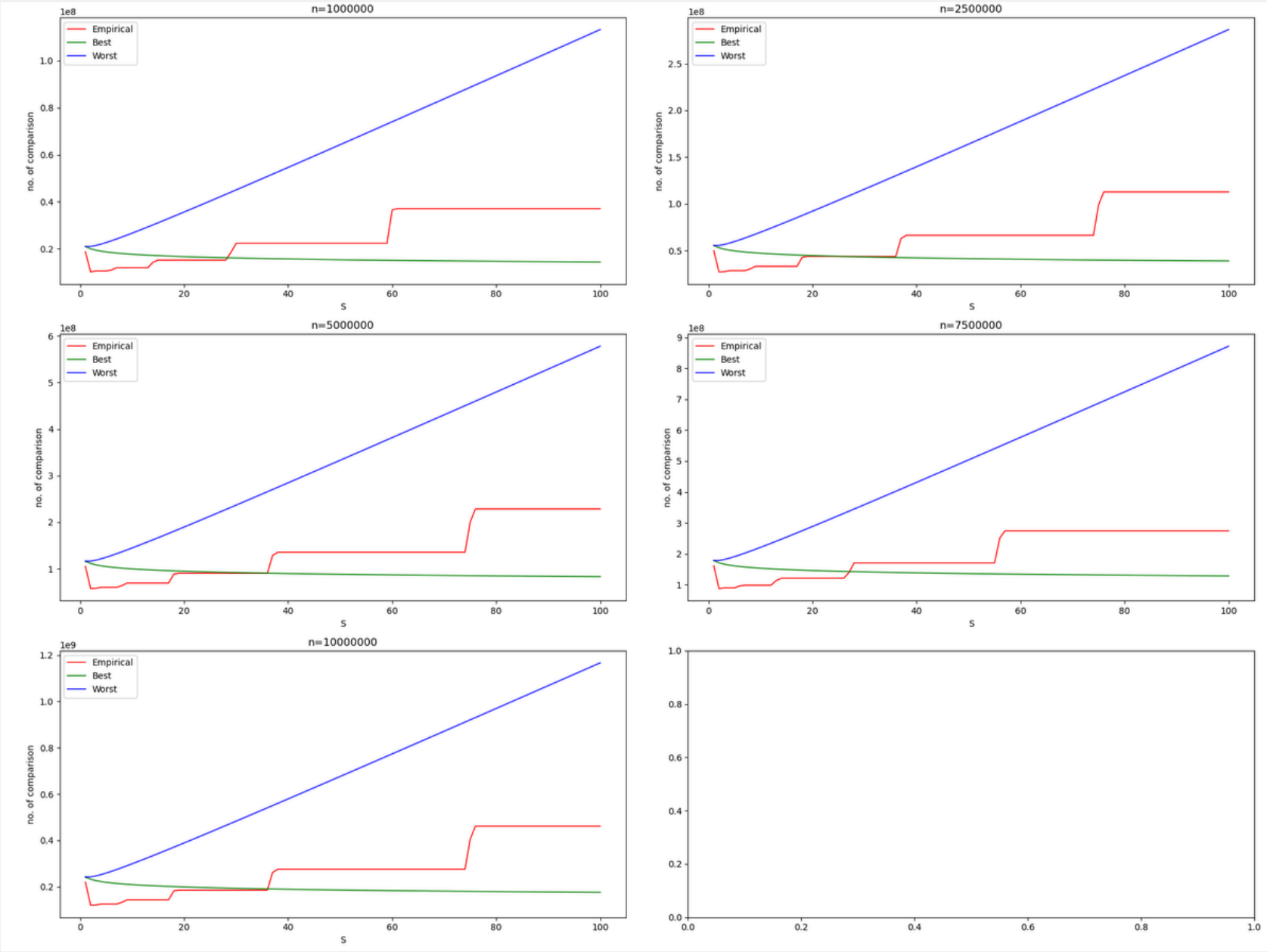
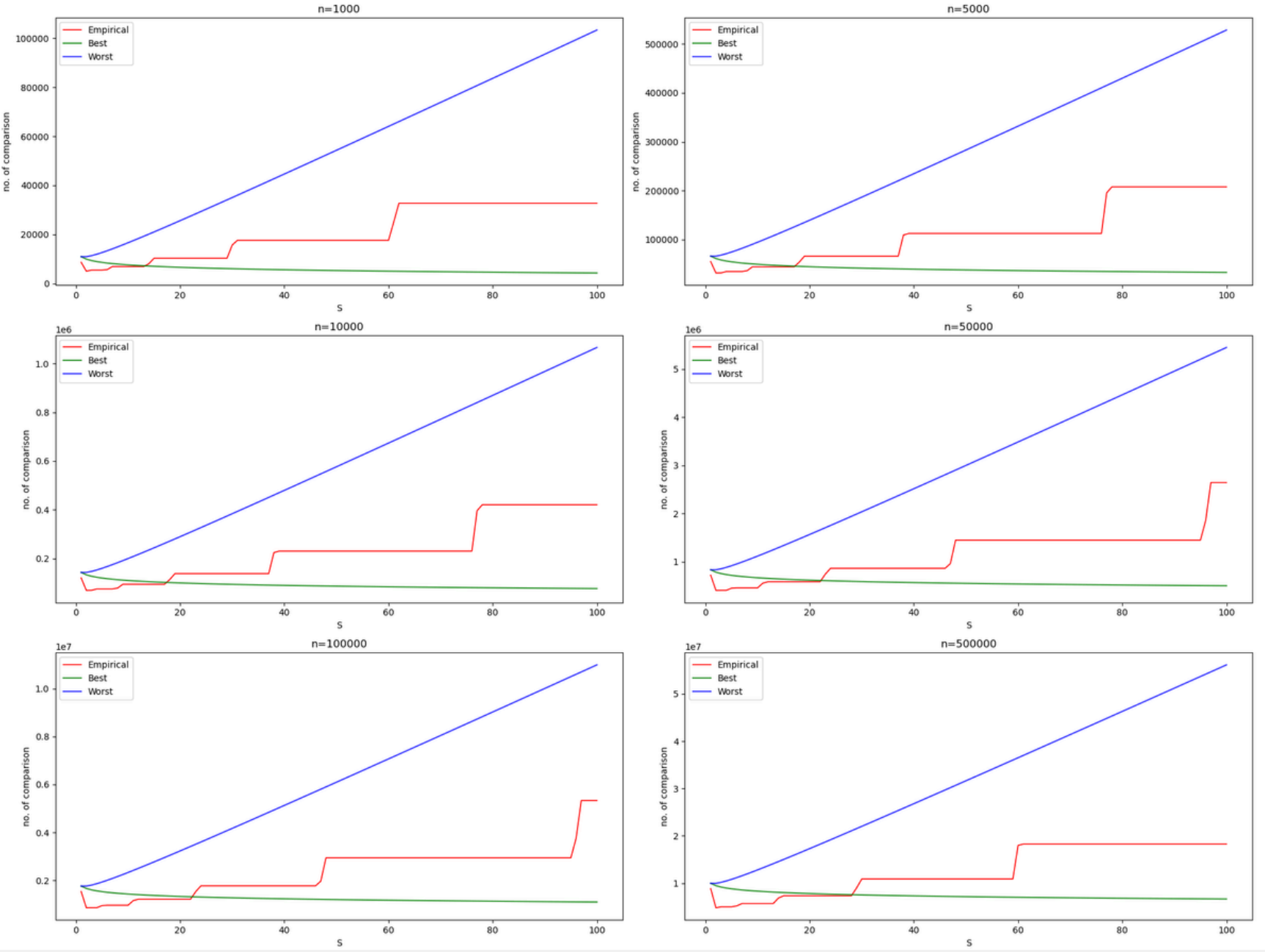
similar result throughout the different S values



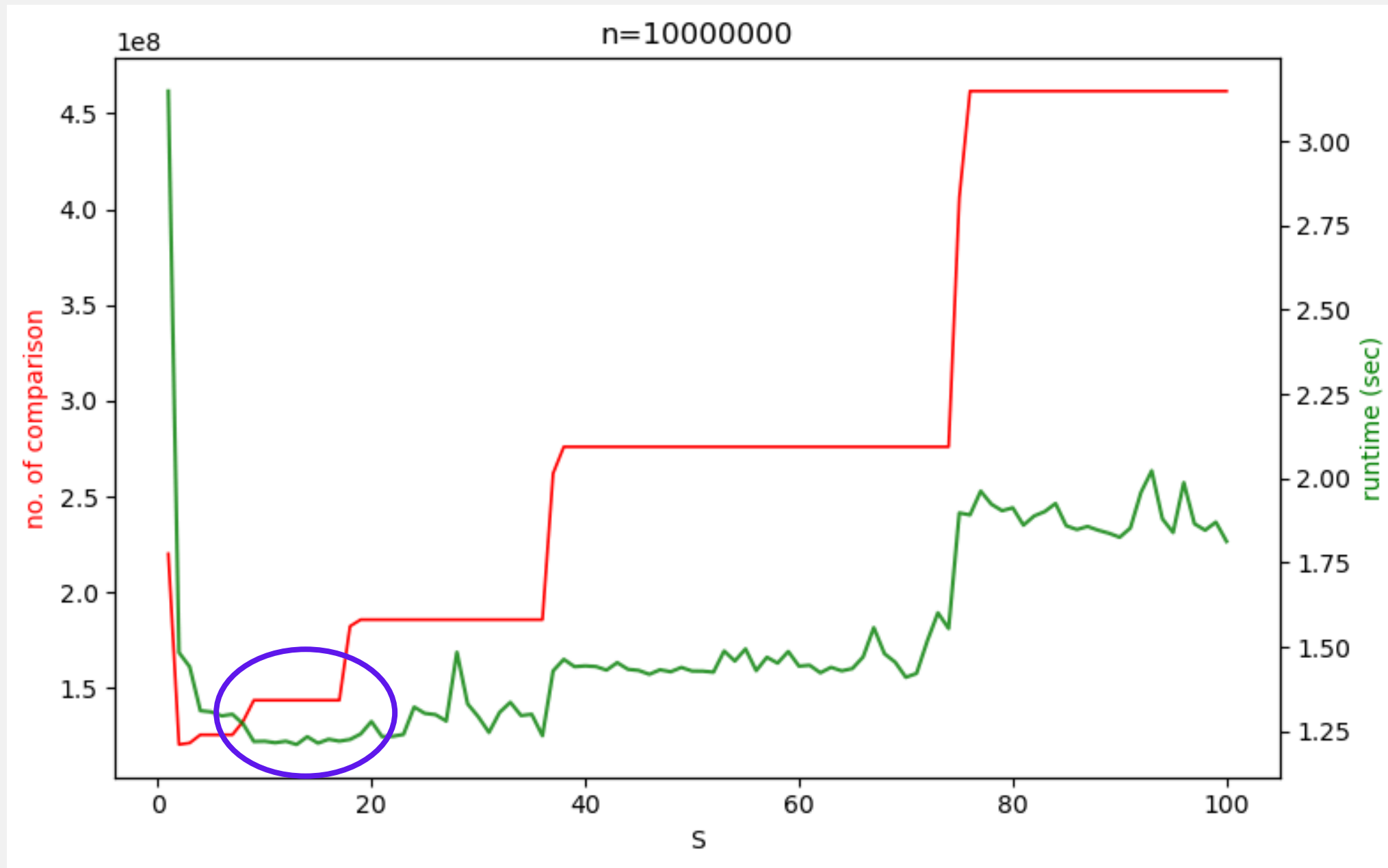
(cii) compare empirical vs theoretical



cii) No. of key comparisons over different values of n



(ciii) find optimal S value

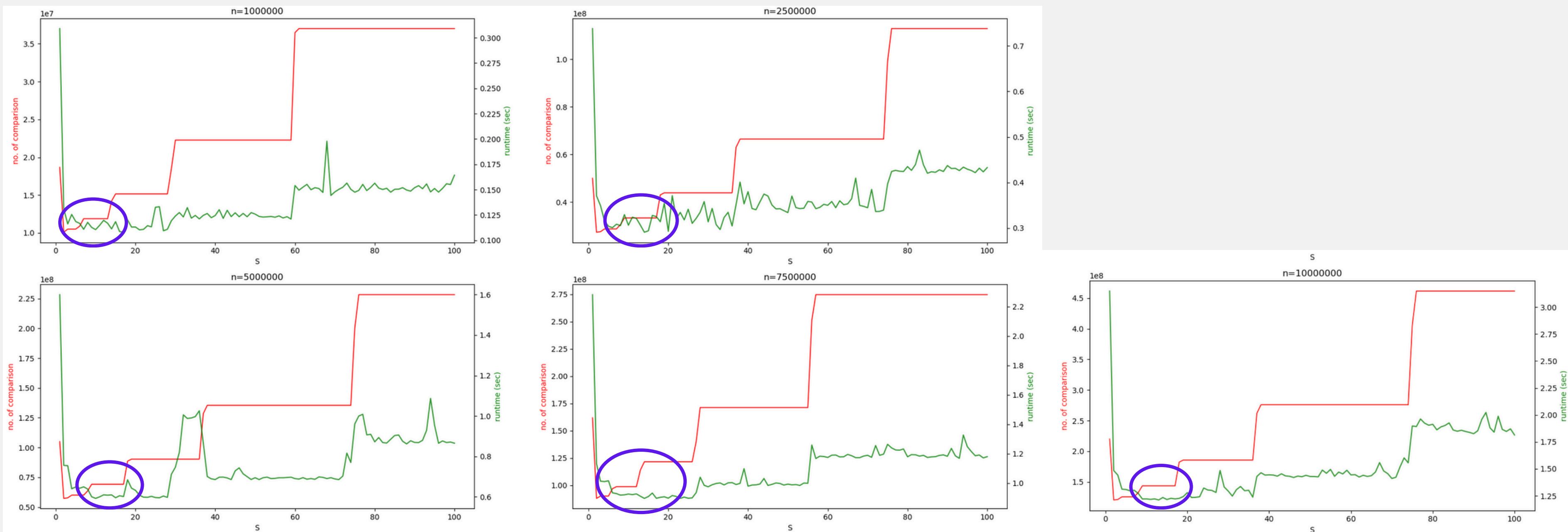


Sometimes, lower comparison but higher runtime

We decided that we should focus on faster runtime to find optimal S value

From our data, we found out that the fastest runtime is around $S < 20$, where comparison is also quite low

similar pattern could be found in the other values of n



```
In [95]: optimal_S = []
for i in range(len(df_n)):
    temp_df = df_n[i][['S', 'runtime', 'comparison']][:21]
    temp_df = temp_df.sort_values(by=['runtime']).reset_index()
    optimal_S.append(temp_df['S'][0])

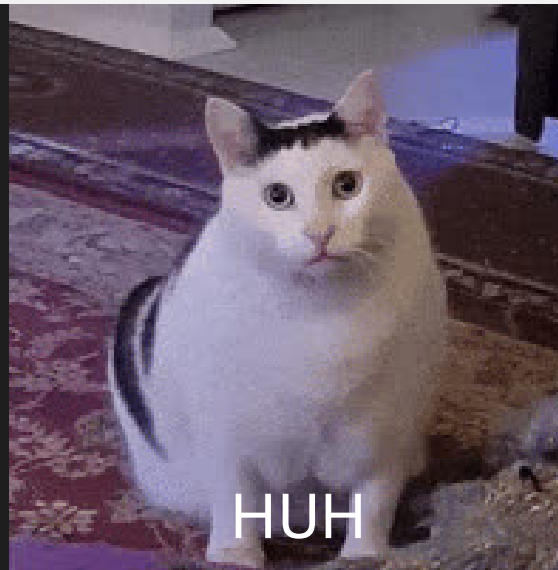
print(optimal_S)
avg_S = sum(optimal_S) // len(optimal_S)
print("optimal S:", avg_S)

[19, 5, 5, 14, 21, 17, 17, 14, 10, 14, 13]
optimal S: 13
```

- Take data points for $S < 20$ and filter out the rest
- Sort the data according to runtime
- Get the corresponding S value for the fastest runtime
- Find the average S value
- **Optimal S value = 13**

(d) Compare with Mergesort

```
138  n = 10000000;  
139  S = 13; // optimal value  
140  arr = generateArr(n, x);  
141  comparison = 0;  
  
142  
143  // mergesort  
144  begin = clock();  
145  mergesort(arr, n);  
146  end = clock();  
147  runtime = (double) (end - begin) / CLOCKS_PER_SEC;  
148  int mergesort_comparison = comparison;  
149  float mergesort_runtime = runtime;  
  
150  
151  comparison = 0;  
  
152  
153  // hybrid sort  
154  begin = clock();  
155  hybrid_sort(arr, n, S);  
156  end = clock();  
157  runtime = (double) (end - begin) / CLOCKS_PER_SEC;  
158  int hybrid_comparison = comparison;  
159  float hybrid_runtime = runtime;
```



$n = 10,000,000$

$S = 13$ (optimal value)

1. Generate array of size n
2. Comparison = track number of comparisons
3. Clock() = returns number of clock ticks elapsed since the program was executed
4. Runtime = Difference between the end and begin divided by CLOCKS_PER_SEC
5. Store results into respective variables

(d) Compare with Mergesort

$n = 10,000,000$ &
 $S = 13$ (optimal value)

Comparison:

Hybrid sort $\sim 1/2$ the number of comparisons than mergesort

- Switches to insertion sort $\rightarrow$ avoids repeated merging

Runtime:

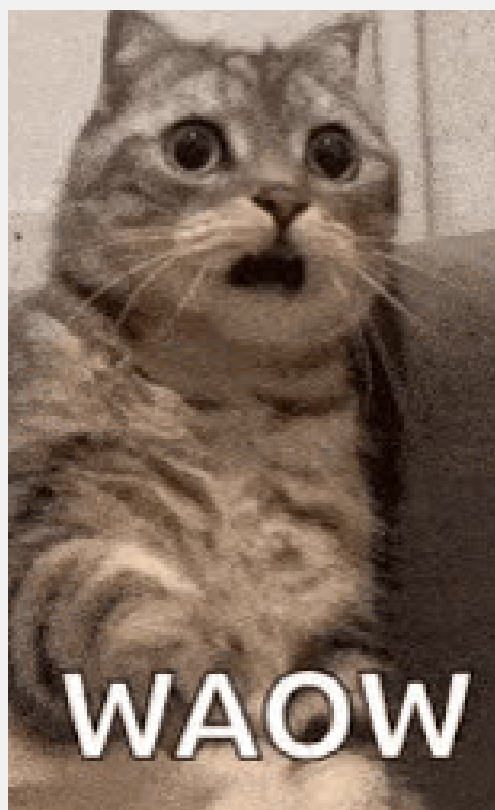
Hybrid sort $\sim 3x$ faster than mergesort

- Insertion sort is faster on smaller arrays $\rightarrow$ reduces time spent on overhead of recursive sorting and merging

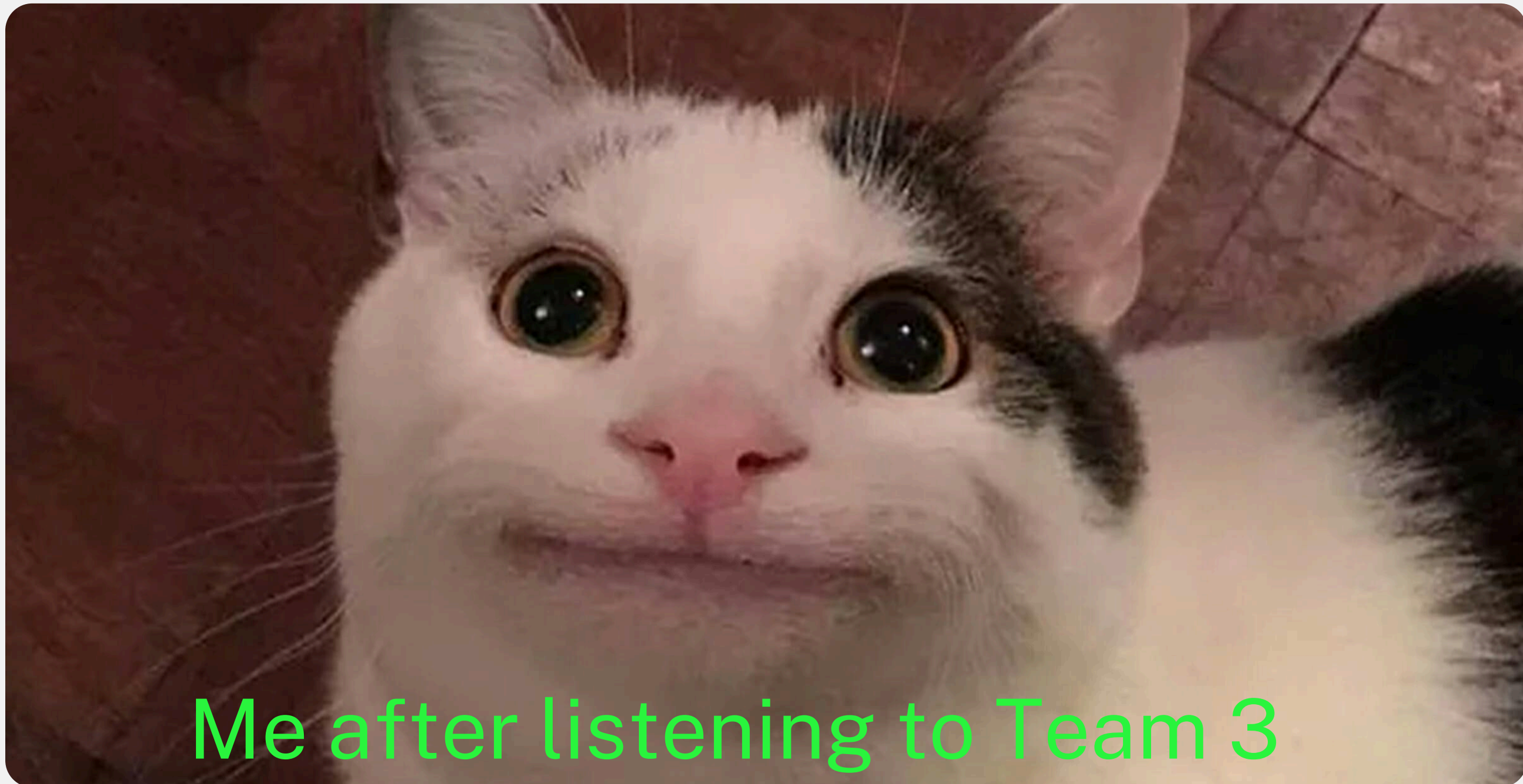
Overall:

Hybrid Sort is more efficient than Mergesort

```
> ./hybrid_merge_sort
      |merge sort |hybrid sort
-----
comparison |220101112 |143660793
runtime    |1.722053 |0.618438
```



Thank You for listening



Me after listening to Team 3